

# Hearsay — source code

UnivaBio 2026 · Sharon Basovich. Full repository: <https://github.com/sharonbasovich/hearsay>. Binary audio stimuli (public/stimuli/\*.wav) omitted.

## README.md

# Hearsay – a 3-minute hearing-in-noise check for families

> "What?" is the most expensive word at dinner.

Hearsay is a browser-based **educational self-check** of how well you understand speech in noise – the ability that usually slips first when hearing starts to change. You hear three spoken digits in noise, type them, and a **Bayesian adaptive engine** picks the next noise level to learn the most from every answer. It stops after about 12 triplets.

Because home headphones are never calibrated, Hearsay **does not show a fake "normal range"**. Instead, it compares family members **on the same device and headphones**, where device offsets cancel, and gently flags anyone who found it noticeably harder than everyone else.

**Hearsay is not a medical device and does not diagnose anything.** It has not been clinically validated.

- Live demo: see the GitHub Pages link in the repo description (or run locally, below)
- Devpost write-up: [ `docs/DEVPOST.md` ](docs/DEVPOST.md)
- One-page summary: [ `docs/hearsay-one-pager.pdf` ](docs/hearsay-one-pager.pdf)
- Data provenance and limitations: [ `docs/PROVENANCE.md` ](docs/PROVENANCE.md)

![[Family board](docs/screenshots/07-family-board.png)]

### ## Features

| Screen                | What it does                                                                                                                                                |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ---                   | ---                                                                                                                                                         |
| <b>Take the check</b> | Volume setup → 6-trial headphone check (antiphase-tone method) → adaptive digits-in-noise test with a live belief chart → result with 90% credible interval |
| <b>Family board</b>   | Groups results by device; compares each person with the median of the others; flags only gaps > 3 dB <i>and</i> larger than the measurement uncertainty     |
| <b>Hear-through</b>   | Plays a sentence and digits through illustrative high-frequency attenuation profiles so relatives can hear why "just listen harder" doesn't work            |
| <b>Lab</b>            | Runs a seeded simulation study in your browser: Hearsay vs. classic staircases on hundreds of virtual listeners                                             |
| <b>Honesty</b>        | What's real, what's synthetic, and what's missing                                                                                                           |

A "Watch a virtual listener" button in the Lab runs the real test UI with a simulated listener answering, so judges can see the full flow without headphones.

### ## Measured results (simulation, `npm run simulate`)

400 virtual listeners, true SRT uniform in [-14, 2] dB, 2% lapse rate, each tested twice:

| Procedure                      | Avg. triplets | RMSE (dB)   | Test-retest SD (dB) | 90% interval coverage |
|--------------------------------|---------------|-------------|---------------------|-----------------------|
| ---                            | ---           | ---         | ---                 | ---                   |
| Staircase, 24 trials (classic) | 24.0          | 0.79        | 1.05                | 62%                   |
| Staircase, 12 trials           | 12.0          | 1.53        | 1.31                | 54%                   |
| <b>Hearsay Bayesian</b>        | <b>11.8</b>   | <b>1.01</b> | <b>1.42</b>         | <b>94%</b>            |

True psychometric slope 0.7/dB. Results at slopes 0.5 and 1.0 are in [ `docs/validation.json` ](docs/validation.json).

Plainly: at the same length Hearsay has ~34% lower error than a staircase and honest error bars; the classic 24-trial staircase is still a bit more precise, at twice the length. These are **simulated listeners, not people**.

### ## How it works

- **Stimuli** – nine digit words (0–9 without the two-syllable "7") synthesized once with ElevenLabs TTS, trimmed and RMS-equalized; stationary speech-shaped noise computed locally from their long-term spectrum ( `scripts/make-stimuli.ts` , `src/dsp/` ).
- **Presentation** – digits are phase-inverted in one ear while the noise is identical in both (antiphasic), as in published antiphasic digits-in-noise designs ( `src/audio/mix.ts` ).
- **Engine** – grid posterior over threshold × slope (3 slopes marginalized), logistic psychometric function with lapses, next SNR chosen to minimize expected posterior entropy of the threshold, stop at posterior SD ≤ 1.25 dB (min 10, max 24 trials) ( `src/engine/bayes.ts` ).
- **Family comparison** – leave-one-out median; verdict requires  $\delta > \max(3 \text{ dB}, 1.645 \cdot \text{hypot}(\text{sd}_i, \text{sd}_{\text{ref}}))$ ; mathematically invariant to a shared device offset ( `src/engine/family.ts` , tested).
- **Privacy** – no backend; results live in `localStorage` only.

### ## Run it

```
``bash
npm install
npm run dev      # http://localhost:5173
npm test        # 32 unit tests (engine, DSP, mixing, family logic, shipped stimuli)
npm run typecheck
npm run build    # static site in dist/
npm run simulate # regenerates docs/validation.json
```

Regenerating the speech (`npm run stimuli`) needs an `ELEVENLABS\_API\_KEY`; the generated WAVs are already committed.

## License

Code: MIT. Speech audio: synthesized with [ElevenLabs](https://elevenlabs.io); see [`docs/PROVENANCE.md`](docs/PROVENANCE.md).

## package.json

```
{
  "name": "hearsay",
  "version": "0.1.0",
  "private": true,
  "description": "A 3-minute digits-in-noise hearing check with a Bayesian adaptive engine, same-device family comparison, and a hear-through simulator. Screening-style self-check, not a diagnosis.",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview",
    "test": "vitest run",
    "typecheck": "tsc --noEmit",
    "stimuli": "vite-node scripts/make-stimuli.ts",
    "simulate": "vite-node scripts/simulate.ts"
  },
  "devDependencies": {
    "@types/node": "20.16.10",
    "typescript": "5.6.3",
    "vite": "5.4.8",
    "vite-node": "2.1.2",
    "vitest": "2.1.2"
  }
}
```

## vite.config.ts

```
import { defineConfig } from "vitest/config";

export default defineConfig({
  base: "./",
  build: {
    target: "es2022",
  },
  test: {
    environment: "node",
    include: ["tests/**/*.test.ts"],
    testTimeout: 120_000,
  },
});
```

## index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Hearsay: a 3-minute digits-in-noise hearing check with a Bayesian adaptive engine and same-device family comparison. Not a diagnosis." />
    <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'%3E%3Ctext y='.9em' font-size='90'%3EF0%9F%91%82%3C/text%3E%3C/svg%3E" />
    <title>Hearsay – a 3-minute hearing-in-noise check</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

## src/audio/hearloss.ts

```
/** Audiogram frequencies (Hz) used by the hear-through simulator. */
export const AUDIOGRAM_FREQS = [250, 500, 1000, 2000, 4000, 8000] as const;

export interface Profile {
  id: string;
  label: string;
  description: string;
  /** Attenuation in dB at each AUDIOGRAM_FREQS entry. Illustrative only. */
  loss: number[];
}
```

```

}

export const PROFILES: Profile[] = [
  { id: "none", label: "Typical", description: "No filtering.", loss: [0, 0, 0, 0, 0, 0] },
  {
    id: "mild-hf",
    label: "Mild high-frequency",
    description: "Illustrative gentle slope above 2 kHz – consonants like s, f and th start to blur.",
    loss: [0, 0, 5, 20, 35, 45],
  },
  {
    id: "moderate-sloping",
    label: "Moderate sloping",
    description: "Illustrative steeper slope – vowels stay loud, so speech sounds 'there' but unclear.",
    loss: [10, 15, 30, 45, 60, 70],
  },
  {
    id: "noise-notch",
    label: "Noise notch",
    description: "Illustrative dip around 4 kHz, a pattern often discussed with long-term loud-noise exposure.",
    loss: [0, 0, 5, 15, 40, 20],
  },
];

/** Linear interpolation (in log-frequency) of a profile's attenuation at frequency f. */
export function lossAt(profile: Profile, f: number): number {
  const fs = AUDIOGRAM_FREQS;
  if (f <= fs[0]) return profile.loss[0];
  if (f >= fs[fs.length - 1]) return profile.loss[fs.length - 1];
  for (let i = 0; i < fs.length - 1; i++) {
    if (f >= fs[i] && f <= fs[i + 1]) {
      const t = Math.log(f / fs[i]) / Math.log(fs[i + 1] / fs[i]);
      return profile.loss[i] + t * (profile.loss[i + 1] - profile.loss[i]);
    }
  }
  return 0;
}

```

## src/audio/mix.ts

```

import { dbToGain } from "../dsp/fft";

export interface MixOptions {
  sampleRate: number;
  snr: number;
  /** Invert the speech in the right ear (antiphasic presentation); noise stays identical in both ears. */
  antiphasic: boolean;
  noiseGain?: number;
  leadMs?: number;
  gapMs?: number;
  tailMs?: number;
  noiseOffset?: number;
}

export interface StereoMix {
  left: Float32Array;
  right: Float32Array;
  onsets: number[];
}

/** Mix three digit tokens into looping speech-shaped noise at the requested SNR. */
export function mixTriplet(tokens: Float32Array[], noise: Float32Array, o: MixOptions): StereoMix {
  const sr = o.sampleRate;
  const noiseGain = o.noiseGain ?? 0.35;
  const lead = Math.round(((o.leadMs ?? 500) / 1000) * sr);
  const gap = Math.round(((o.gapMs ?? 300) / 1000) * sr);
  const tail = Math.round(((o.tailMs ?? 400) / 1000) * sr);
  const len = lead + tokens.reduce((a, t) => a + t.length, 0) + gap * (tokens.length - 1) + tail;
  const left = new Float32Array(len);
  const right = new Float32Array(len);
  const off = o.noiseOffset ?? 0;
  const ramp = Math.round(0.02 * sr);
  for (let i = 0; i < len; i++) {
    const env = Math.min(1, i / ramp, (len - 1 - i) / ramp);
    const n = noise[(i + off) % noise.length] * noiseGain * env;
    left[i] = n;
    right[i] = n;
  }
  const sg = noiseGain * dbToGain(o.snr);
  const onsets: number[] = [];
  let pos = lead;
  for (const t of tokens) {
    onsets.push(pos / sr);
    for (let i = 0; i < t.length; i++) {
      left[pos + i] += t[i] * sg;
    }
  }
}

```

```

    right[pos + i] += (o.antiphasic ? -1 : 1) * t[i] * sg;
  }
  pos += t.length + gap;
}
let peak = 0;
for (let i = 0; i < len; i++) peak = Math.max(peak, Math.abs(left[i]), Math.abs(right[i]));
if (peak > 0.95) {
  const g = 0.95 / peak;
  for (let i = 0; i < len; i++) {
    left[i] *= g;
    right[i] *= g;
  }
}
return { left, right, onsets };
}

```

## src/audio/player.ts

```

import { mixTriplet } from "../mix";
import { AUDIOGRAM_FREQS, type Profile } from "../hearloss";
import type { Digit } from "../engine/triplets";

export class AudioEngine {
  readonly ctx: AudioContext;
  readonly master: GainNode;
  private tokens = new Map<number, Float32Array>();
  private noise = new Float32Array(0);
  private sentence: AudioBuffer | null = null;
  private current: AudioBufferSourceNode | null = null;
  private loaded: Promise<void> | null = null;
  private noiseOffset = 0;

  constructor() {
    this.ctx = new AudioContext();
    this.master = this.ctx.createGain();
    this.master.gain.value = 0.5;
    this.master.connect(this.ctx.destination);
  }

  load(base = "../stimuli/"): Promise<void> {
    this.loaded ??= (async () => {
      const get = async (name: string) => {
        const res = await fetch(base + name);
        if (!res.ok) throw new Error(`Could not load ${name}`);
        return this.ctx.decodeAudioData(await res.arrayBuffer());
      };
      const digits: Digit[] = [0, 1, 2, 3, 4, 5, 6, 8, 9];
      await Promise.all(digits.map(async (d) => this.tokens.set(d, (await get(`digit-${d}.wav`)).getChannelData(0))));
      this.noise = (await get("noise.wav")).getChannelData(0);
      this.sentence = await get("sentence.wav");
    })();
    return this.loaded;
  }

  setVolume(v: number): void {
    this.master.gain.setTargetAtTime(v, this.ctx.currentTime, 0.02);
  }

  stop(): void {
    try {
      this.current?.stop();
    } catch {
      /* already stopped */
    }
    this.current = null;
  }

  private play(buf: AudioBuffer, dest: AudioNode = this.master): Promise<void> {
    this.stop();
    const src = this.ctx.createBufferSource();
    src.buffer = buf;
    src.connect(dest);
    this.current = src;
    return new Promise((resolve) => {
      src.onended = () => resolve();
      src.start();
    });
  }

  async playTriplet(digits: readonly Digit[], snr: number, antiphasic = true, dest?: AudioNode): Promise<void> {
    await this.ctx.resume();
    const mix = mixTriplet(
      digits.map((d) => this.tokens.get(d)!),
      this.noise,
      { sampleRate: this.ctx.sampleRate, snr, antiphasic, noiseOffset: this.noiseOffset },
    );
  }
}

```

```

    );
    this.noiseOffset = (this.noiseOffset + 9973) % Math.max(1, this.noise.length);
    const buf = this.ctx.createBuffer(2, mix.left.length, this.ctx.sampleRate);
    buf.copyToChannel(mix.left, 0);
    buf.copyToChannel(mix.right, 1);
    return this.play(buf, dest);
}

/** Three 200 Hz tones; `quiet` is 6 dB softer, `antiphase` is phase-inverted in the right ear. */
async playHeadphoneTrial(quiet: number, antiphase: number): Promise<void> {
    await this.ctx.resume();
    const sr = this.ctx.sampleRate;
    const toneLen = Math.round(sr * 0.8);
    const gapLen = Math.round(sr * 0.5);
    const len = toneLen * 3 + gapLen * 2;
    const buf = this.ctx.createBuffer(2, len, sr);
    const l = buf.getChannelData(0);
    const r = buf.getChannelData(1);
    for (let t = 0; t < 3; t++) {
        const amp = 0.3 * (t === quiet ? 0.5 : 1);
        const start = t * (toneLen + gapLen);
        for (let i = 0; i < toneLen; i++) {
            const env = Math.min(1, i / (0.05 * sr), (toneLen - i) / (0.05 * sr));
            const v = amp * env * Math.sin((2 * Math.PI * 200 * i) / sr);
            l[start + i] = v;
            r[start + i] = t === antiphase ? -v : v;
        }
    }
    return this.play(buf);
}

/** Chain of peaking filters approximating an illustrative attenuation profile. */
lossChain(profile: Profile): AudioNode {
    let head: AudioNode = this.master;
    const nodes = [...AUDIOGRAM_FREQS].reverse().map((f, idx) => {
        const i = AUDIOGRAM_FREQS.length - 1 - idx;
        const bq = this.ctx.createBiquadFilter();
        if (i === AUDIOGRAM_FREQS.length - 1) {
            bq.type = "highshelf";
            bq.frequency.value = f * 0.75;
        } else {
            bq.type = "peaking";
            bq.frequency.value = f;
            bq.Q.value = 1.0;
        }
        bq.gain.value = -profile.loss[i];
        return bq;
    });
    for (const n of nodes) {
        n.connect(head);
        head = n;
    }
    return head;
}

async playSentence(profile: Profile | null): Promise<void> {
    await this.ctx.resume();
    if (!this.sentence) return;
    return this.play(this.sentence, profile ? this.lossChain(profile) : this.master);
}

async playTripletThrough(profile: Profile, digits: readonly Digit[], snr: number): Promise<void> {
    return this.playTriplet(digits, snr, false, this.lossChain(profile));
}
}

```

## src/dsp/fft.ts

```

export function fft(re: Float64Array, im: Float64Array, inverse = false): void {
    const n = re.length;
    if (n !== im.length || (n & (n - 1)) !== 0) throw new Error("FFT length must be a power of two");
    for (let i = 1, j = 0; i < n; i++) {
        let bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) {
            [re[i], re[j]] = [re[j], re[i]];
            [im[i], im[j]] = [im[j], im[i]];
        }
    }
    for (let len = 2; len <= n; len <= 1) {
        const ang = ((inverse ? 2 : -2) * Math.PI) / len;
        const wr = Math.cos(ang);
        const wi = Math.sin(ang);
        for (let i = 0; i < n; i += len) {

```

```

    let cr = 1;
    let ci = 0;
    for (let k = 0; k < len / 2; k++) {
        const a = i + k;
        const b = a + len / 2;
        const tr = re[b] * cr - im[b] * ci;
        const ti = re[b] * ci + im[b] * cr;
        re[b] = re[a] - tr;
        im[b] = im[a] - ti;
        re[a] += tr;
        im[a] += ti;
        const ncr = cr * wr - ci * wi;
        ci = cr * wi + ci * wr;
        cr = ncr;
    }
}
}
if (inverse) {
    for (let i = 0; i < n; i++) {
        re[i] /= n;
        im[i] /= n;
    }
}
}

export function rms(x: ArrayLike<number>): number {
    let s = 0;
    for (let i = 0; i < x.length; i++) s += x[i] * x[i];
    return Math.sqrt(s / Math.max(1, x.length));
}

export function dbToGain(db: number): number {
    return Math.pow(10, db / 20);
}

export function nextPow2(n: number): number {
    let p = 1;
    while (p < n) p <= 1;
    return p;
}

```

## src/dsp/stimuli.ts

```

import { fft, nextPow2, rms } from "../fft";
import { gaussian, mulberry32 } from "../engine/rng";

export const TARGET_RMS = 0.1;

export function trimSilence(x: Float32Array, sampleRate: number, thresholdDb = -40, padMs = 15): Float32Array {
    let peak = 0;
    for (let i = 0; i < x.length; i++) peak = Math.max(peak, Math.abs(x[i]));
    if (peak === 0) return x.slice(0, 0);
    const thr = peak * Math.pow(10, thresholdDb / 20);
    let start = 0;
    while (start < x.length && Math.abs(x[start]) < thr) start++;
    let end = x.length - 1;
    while (end > start && Math.abs(x[end]) < thr) end--;
    const pad = Math.round((padMs / 1000) * sampleRate);
    return x.slice(Math.max(0, start - pad), Math.min(x.length, end + pad + 1));
}

export function normalizeRms(x: Float32Array, target = TARGET_RMS): Float32Array {
    const r = rms(x);
    if (r === 0) return x.slice();
    const g = target / r;
    const out = new Float32Array(x.length);
    for (let i = 0; i < x.length; i++) out[i] = x[i] * g;
    return out;
}

export function averagePowerSpectrum(signals: Float32Array[], n: number): Float64Array {
    const power = new Float64Array(n);
    for (const s of signals) {
        const re = new Float64Array(n);
        const im = new Float64Array(n);
        for (let i = 0; i < Math.min(n, s.length); i++) re[i] = s[i];
        fft(re, im);
        for (let k = 0; k < n; k++) power[k] += re[k] * re[k] + im[k] * im[k];
    }
    for (let k = 0; k < n; k++) power[k] /= Math.max(1, signals.length);
    return power;
}

/** Stationary noise whose long-term spectrum matches the speech tokens; circular so it loops seamlessly. */
export function speechShapedNoise(signals: Float32Array[], length: number, seed: number, target = TARGET_RMS): Float32

```

```

Array {
  const n = nextPow2(length);
  const specN = nextPow2(Math.max(...signals.map((s) => s.length)));
  const p = averagePowerSpectrum(signals, specN);
  const rng = mulberry32(seed);
  const re = new Float64Array(n);
  const im = new Float64Array(n);
  for (let k = 0; k <= n / 2; k++) {
    const src = Math.min(specN - 1, Math.round((k / n) * specN));
    const mag = Math.sqrt(p[src]);
    const a = gaussian(rng);
    const b = gaussian(rng);
    re[k] = mag * a;
    im[k] = k === 0 || k === n / 2 ? 0 : mag * b;
    if (k > 0 && k < n / 2) {
      re[n - k] = re[k];
      im[n - k] = -im[k];
    }
  }
  fft(re, im, true);
  const out = new Float32Array(n);
  for (let i = 0; i < n; i++) out[i] = re[i];
  return normalizeRms(out, target);
}

```

## src/engine/bayes.ts

```

import { DEFAULT_LAPSE, pCorrect } from "../psychometric";
import type { AdaptiveProcedure, Estimate, Trial } from "../types";

export interface BayesOptions {
  srtMin?: number;
  srtMax?: number;
  srtStep?: number;
  slopes?: number[];
  priorMean?: number;
  priorSd?: number;
  lapse?: number;
  candidates?: number[];
  minTrials?: number;
  maxTrials?: number;
  targetSd?: number;
}

function range(a: number, b: number, step: number): number[] {
  const out: number[] = [];
  for (let x = a; x <= b + 1e-9; x += step) out.push(Math.round(x * 1000) / 1000);
  return out;
}

/**
 * Grid-based Bayesian adaptive procedure (in the spirit of QUEST+/psi).
 * Joint posterior over SRT and psychometric slope; each trial is placed at the SNR that
 * minimizes the expected posterior entropy of the SRT, and the run stops once the
 * SRT posterior is tight enough.
 */
export class BayesianDin implements AdaptiveProcedure {
  readonly name = "Hearsay Bayesian";
  readonly srts: number[];
  readonly slopes: number[];
  readonly candidates: number[];
  private post: Float64Array;
  private readonly o: Required<BayesOptions>;
  private readonly lik: Float64Array[];
  private n = 0;

  constructor(opts: BayesOptions = {}) {
    this.o = {
      srtMin: -22,
      srtMax: 10,
      srtStep: 0.25,
      slopes: [0.4, 0.7, 1.1],
      priorMean: -8,
      priorSd: 7,
      lapse: DEFAULT_LAPSE,
      candidates: range(-22, 10, 1),
      minTrials: 10,
      maxTrials: 24,
      targetSd: 1.25,
      ...opts,
    };
    this.srts = range(this.o.srtMin, this.o.srtMax, this.o.srtStep);
    this.slopes = this.o.slopes;
    this.candidates = this.o.candidates;
    const S = this.srts.length;

```

```

const K = this.slopes.length;
this.post = new Float64Array(S * K);
for (let i = 0; i < S; i++) {
  const z = (this.srts[i] - this.o.priorMean) / this.o.priorSd;
  for (let k = 0; k < K; k++) this.post[i * K + k] = Math.exp(-0.5 * z * z) / K;
}
normalize(this.post);
this.lik = this.candidates.map((snr) => {
  const l = new Float64Array(S * K);
  for (let i = 0; i < S; i++) for (let k = 0; k < K; k++) l[i * K + k] = pCorrect(snr, this.srts[i], this.slopes
[k], this.o.lapse);
  return l;
});
}

private srtMarginal(p: Float64Array): Float64Array {
  const K = this.slopes.length;
  const m = new Float64Array(this.srts.length);
  for (let i = 0; i < m.length; i++) for (let k = 0; k < K; k++) m[i] += p[i * K + k];
  return m;
}

nextSnr(): number {
  let best = 0;
  let bestH = Infinity;
  const tmp = new Float64Array(this.post.length);
  for (let c = 0; c < this.candidates.length; c++) {
    const l = this.lik[c];
    let pc = 0;
    for (let j = 0; j < l.length; j++) pc += this.post[j] * l[j];
    let h = 0;
    for (const outcome of [true, false]) {
      const po = outcome ? pc : 1 - pc;
      if (po < 1e-9) continue;
      for (let j = 0; j < l.length; j++) tmp[j] = (this.post[j] * (outcome ? l[j] : 1 - l[j])) / po;
      h += po * entropy(this.srtMarginal(tmp));
    }
    if (h < bestH - 1e-12) {
      bestH = h;
      best = c;
    }
  }
  return this.candidates[best];
}

update(trial: Trial): void {
  const K = this.slopes.length;
  for (let i = 0; i < this.srts.length; i++) {
    for (let k = 0; k < K; k++) {
      const p = pCorrect(trial.snr, this.srts[i], this.slopes[k], this.o.lapse);
      this.post[i * K + k] *= trial.correct ? p : 1 - p;
    }
  }
  normalize(this.post);
  this.n++;
}

done(): boolean {
  if (this.n >= this.o.maxTrials) return true;
  return this.n >= this.o.minTrials && this.estimate().sd <= this.o.targetSd;
}

/** SRT marginal posterior (for plotting). */
marginal(): { srt: number; p: number }[] {
  const m = this.srtMarginal(this.post);
  return this.srts.map((srt, i) => ({ srt, p: m[i] }));
}

estimate(): Estimate {
  const m = this.srtMarginal(this.post);
  let mean = 0;
  for (let i = 0; i < m.length; i++) mean += m[i] * this.srts[i];
  let v = 0;
  for (let i = 0; i < m.length; i++) v += m[i] * (this.srts[i] - mean) ** 2;
  return { srt: mean, sd: Math.sqrt(v), lo90: quantile(m, this.srts, 0.05), hi90: quantile(m, this.srts, 0.95), tria
ls: this.n };
}

function normalize(p: Float64Array): void {
  let s = 0;
  for (let i = 0; i < p.length; i++) s += p[i];
  for (let i = 0; i < p.length; i++) p[i] /= s;
}

function entropy(p: Float64Array): number {
  let h = 0;

```



```

    for (let i = 0; i < p.length; i++) if (p[i] > 0) h -= p[i] * Math.log(p[i]);
    return h;
}

function quantile(p: Float64Array, xs: number[], q: number): number {
    let c = 0;
    for (let i = 0; i < p.length; i++) {
        c += p[i];
        if (c >= q) return xs[i];
    }
    return xs[xs.length - 1];
}

```

## src/engine/family.ts

```

import type { Estimate } from "../types";

export interface Member {
    id: string;
    name: string;
    estimate: Estimate;
    takenAt: string;
    device: string;
}

export type Verdict = "similar" | "harder" | "easier" | "too-few";

export interface Comparison {
    member: Member;
    reference: number;
    delta: number;
    verdict: Verdict;
}

/** Minimum gap (dB) before Hearsay calls a difference notable, regardless of measured precision. */
export const MIN_NOTABLE_DB = 3;

export function median(xs: number[]): number {
    const s = [...xs].sort((a, b) => a - b);
    const m = s.length >> 1;
    return s.length % 2 ? s[m] : (s[m - 1] + s[m]) / 2;
}

/**
 * Compare each member with the median of everyone else tested on the same device and headphones.
 * Because everyone shares the same (uncalibrated) output chain, device-level offsets cancel out.
 * A difference is notable only if it exceeds both MIN_NOTABLE_DB and 1.645x the combined uncertainty.
 */
export function compareGroup(members: Member[]): Comparison[] {
    return members.map((member) => {
        const others = members.filter((m) => m.id !== member.id);
        if (others.length === 0) return { member, reference: NaN, delta: NaN, verdict: "too-few" as const };
        const reference = median(others.map((m) => m.estimate.srt));
        const refSd = median(others.map((m) => m.estimate.sd));
        const delta = member.estimate.srt - reference;
        const threshold = Math.max(MIN_NOTABLE_DB, 1.645 * Math.hypot(member.estimate.sd, refSd));
        const verdict: Verdict = delta > threshold ? "harder" : delta < -threshold ? "easier" : "similar";
        return { member, reference, delta, verdict };
    });
}

```

## src/engine/headphones.ts

```

import type { Rng } from "../rng";

/**
 * Headphone screening in the style of Woods et al. (2017): three 200 Hz tones, one of which is
 * presented in antiphase between the ears. Over headphones the antiphase tone is not quieter;
 * the target is the one that is actually 6 dB quieter. Over loudspeakers the antiphase tone
 * partially cancels acoustically and tends to be picked instead.
 */
export interface HeadphoneTrial {
    quiet: number;
    antiphase: number;
}

export function makeHeadphoneTrial(rng: Rng): HeadphoneTrial {
    const quiet = Math.floor(rng() * 3);
    let antiphase = Math.floor(rng() * 2);
    if (antiphase >= quiet) antiphase++;
    return { quiet, antiphase };
}

export const HEADPHONE_TRIALS = 6;

```

```
export const HEADPHONE_PASS = 5;

export function headphonePassed(correct: number): boolean {
  return correct >= HEADPHONE_PASS;
}
```

## src/engine/psychometric.ts

```
/** Probability that a listener repeats a whole digit triplet correctly at a given SNR (dB). */
export interface Listener {
  srt: number;
  slope: number;
  lapse: number;
}

export const DEFAULT_SLOPE = 0.7;
export const DEFAULT_LAPSE = 0.03;

export function pCorrect(snr: number, srt: number, slope = DEFAULT_SLOPE, lapse = DEFAULT_LAPSE): number {
  return (1 - lapse) / (1 + Math.exp(-slope * (snr - srt)));
}
```

## src/engine/rng.ts

```
export type Rng = () => number;

export function mulberry32(seed: number): Rng {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6d2b79f5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

export function gaussian(rng: Rng): number {
  const u = Math.max(rng(), 1e-12);
  const v = rng();
  return Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
}
```

## src/engine/simulate.ts

```
import { BayesianDin } from "./bayes";
import { pCorrect } from "./psychometric";
import { mulberry32 } from "./rng";
import { Staircase } from "./staircase";
import type { AdaptiveProcedure } from "./types";

export interface SimConfig {
  listeners: number;
  seed: number;
  trueSlope: number;
  trueLapse: number;
  srtRange: [number, number];
}

export interface MethodStats {
  method: string;
  rmse: number;
  bias: number;
  meanTrials: number;
  /** Fraction of runs whose reported 90% interval contained the true SRT. */
  coverage90: number;
  /** SD of the difference between two independent runs on the same simulated listener. */
  retestSd: number;
  errors: number[];
  trials: number[];
}

export const DEFAULT_SIM: SimConfig = { listeners: 400, seed: 7, trueSlope: 0.7, trueLapse: 0.02, srtRange: [-14, 2] };

export type ProcedureFactory = () => AdaptiveProcedure;

export const METHODS: Record<string, ProcedureFactory> = {
  staircase: () => new Staircase(),
  staircase12: () => new Staircase({ trials: 12, discard: 2 }),
  bayesian: () => new BayesianDin(),
};
```

```

export function runOnce(make: ProcedureFactory, srt: number, slope: number, lapse: number, rng: () => number) {
  const proc = make();
  while (!proc.done()) {
    const snr = proc.nextSnr();
    proc.update({ snr, correct: rng() < pCorrect(snr, srt, slope, lapse) });
  }
  return proc.estimate();
}

export function simulate(make: ProcedureFactory, cfg: SimConfig = DEFAULT_SIM): MethodStats {
  const rng = mulberry32(cfg.seed);
  const errors: number[] = [];
  const trials: number[] = [];
  const diffs: number[] = [];
  let covered = 0;
  let name = "";
  for (let i = 0; i < cfg.listeners; i++) {
    const srt = cfg.srtRange[0] + rng() * (cfg.srtRange[1] - cfg.srtRange[0]);
    const a = runOnce(make, srt, cfg.trueSlope, cfg.trueLapse, rng);
    const b = runOnce(make, srt, cfg.trueSlope, cfg.trueLapse, rng);
    errors.push(a.srt - b.srt);
    trials.push(a.trials);
    diffs.push(a.srt - b.srt);
    if (a.lo90 <= srt && srt <= a.hi90) covered++;
    name += make().name;
  }
  const mean = (x: number[]) => x.reduce((s, v) => s + v, 0) / x.length;
  const md = mean(diffs);
  return {
    method: name,
    rmse: Math.sqrt(mean(errors.map((e) => e * e))),
    bias: mean(errors),
    meanTrials: mean(trials),
    coverage90: covered / cfg.listeners,
    retestSd: Math.sqrt(mean(diffs.map((d) => (d - md) ** 2))),
    errors,
    trials,
  };
}

```

## src/engine/staircase.ts

```

import type { AdaptiveProcedure, Estimate, Trial } from "../types";

export interface StaircaseOptions {
  start?: number;
  step?: number;
  trials?: number;
  discard?: number;
  min?: number;
  max?: number;
}

/**
 * Classic fixed-length digits-in-noise track: 1-up/1-down with 2 dB steps.
 * The SRT is the mean SNR of trials after `discard`, plus the level the next trial would have used.
 */
export class Staircase implements AdaptiveProcedure {
  readonly name: string;
  private snr: number;
  private readonly history: Trial[] = [];
  private readonly o: Required<StaircaseOptions>;

  constructor(opts: StaircaseOptions = {}) {
    this.o = { start: 0, step: 2, trials: 24, discard: 4, min: -24, max: 12, ...opts };
    this.snr = this.o.start;
    this.name = `Staircase (${this.o.trials} trials)`;
  }

  nextSnr(): number {
    return this.snr;
  }

  update(trial: Trial): void {
    this.history.push(trial);
    const next = trial.snr + (trial.correct ? -this.o.step : this.o.step);
    this.snr = Math.max(this.o.min, Math.min(this.o.max, next));
  }

  done(): boolean {
    return this.history.length >= this.o.trials;
  }

  estimate(): Estimate {
    const levels = this.history.slice(this.o.discard).map((t) => t.snr);
  }
}

```

```

    levels.push(this.snr);
    const mean = levels.reduce((a, b) => a + b, 0) / levels.length;
    const variance = levels.reduce((a, b) => a + (b - mean) ** 2, 0) / Math.max(1, levels.length - 1);
    const sd = Math.sqrt(variance / levels.length);
    return { srt: mean, sd, lo90: mean - 1.645 * sd, hi90: mean + 1.645 * sd, trials: this.history.length };
  }
}

```

## src/engine/triplets.ts

```

import type { Rng } from "../rng";

/** "7" is left out, as in several English digits-in-noise designs, because it is the only two-syllable digit. */
export const DIGITS = [0, 1, 2, 3, 4, 5, 6, 8, 9] as const;
export type Digit = (typeof DIGITS)[number];
export type Triplet = [Digit, Digit, Digit];

export function makeTriplet(rng: Rng): Triplet {
  const pool: Digit[] = [...DIGITS];
  const out: Digit[] = [];
  for (let i = 0; i < 3; i++) {
    const idx = Math.floor(rng() * pool.length);
    out.push(pool.splice(idx, 1)[0]);
  }
  return out as Triplet;
}

export function parseResponse(text: string): number[] {
  return [...text.replace(/\D/g, "")].map(Number);
}

export function scoreTriplet(target: Triplet, response: string): { correct: boolean; digitsCorrect: number } {
  const r = parseResponse(response);
  let digitsCorrect = 0;
  for (let i = 0; i < 3; i++) if (r[i] === target[i]) digitsCorrect++;
  return { correct: digitsCorrect === 3 && r.length === 3, digitsCorrect };
}

```

## src/engine/types.ts

```

export interface Trial {
  snr: number;
  correct: boolean;
}

export interface Estimate {
  srt: number;
  /** Uncertainty (posterior SD for Bayesian; SD of tracked SNRs / sqrt(n) for the staircase). */
  sd: number;
  lo90: number;
  hi90: number;
  trials: number;
}

export interface AdaptiveProcedure {
  readonly name: string;
  nextSnr(): number;
  update(trial: Trial): void;
  done(): boolean;
  estimate(): Estimate;
}

```

## src/main.ts

```

import "../style.css";
import { AudioEngine } from "../audio/player";
import { AUDIOGRAM_FREQS, PROFILES } from "../audio/hearloss";
import { BayesianDin } from "../engine/bayes";
import { compareGroup, type Member } from "../engine/family";
import { HEADPHONE_TRIALS, headphonePassed, makeHeadphoneTrial } from "../engine/headphones";
import { pCorrect } from "../engine/psychometric";
import { mulberry32 } from "../engine/rng";
import { DEFAULT_SIM, METHODS, simulate, type MethodStats } from "../engine/simulate";
import { makeTriplet, scoreTriplet, type Triplet } from "../engine/triplets";
import type { Estimate, Trial } from "../engine/types";
import { esc, familyChart, histogram, posteriorChart, profileChart, trackChart } from "../ui/charts";
import { demoFamily, loadMembers, saveMembers } from "../storage";
import { ICONS } from "../ui/icons";

const app = document.getElementById("app")!;
let audio: AudioEngine | null = null;
const engine = () => (audio ??= new AudioEngine());
const rng = mulberry32((Date.now() ^ 0x5e5d) >>> 0);

```

```

const DISCLAIMER =
  "Hearsay is an educational self-check, not a medical device. It cannot diagnose hearing loss or any condition. If you are worried about your hearing, see an audiologist or doctor.";

function shell(active: string, body: string): string {
  const link = (href: string, label: string) => `

```

```

    ${st.virtualSrt !== null ? `
```

```

let snr = proc.nextSnr();
let typed = "";

const render = () => {
  const est = proc.estimate();
  main.innerHTML = `<section class="panel test">
    <div class="testhead"><p class="eyebrow">Step 4 of 4 · Digits in noise · triplet ${trials.length + 1}</p><div class="meter"><span style="width:${Math.min(100, (trials.length / 14) * 100)}%"></span></div>
    <div class="testgrid">
      <div>
        <h2>Type the three digits you heard</h2>
        <p class="fine">Guess if unsure. Every answer teaches the engine something.</p>
        <div class="display" aria-live="polite">${[0, 1, 2].map((k) => `<span class="${typed[k] ? "filled" : ""}">
          ${typed[k]} ?? "."</span>`).join("")}</div>
        <div class="keypad">${[1, 2, 3, 4, 5, 6, 8, 9, 0].map((d) => `<button class="key" data-d="${d}">${d}</button>
          >`).join("")}<button class="key alt" data-d="back"></button><button class="key alt" id="replay">⏮</button><button class="key alt" id="submit">⏭</button>
          ${typed.length === 3 ? "" : "disabled"}>OK</button></div>
      </div>
      <aside class="belief">
        <h3>What Hearsay believes right now</h3>
        ${posteriorChart(proc.marginal(), est)}
        <p class="fine">Shaded band = 90% credible interval for your speech reception threshold (the noise level where you get half the triplets right). Current: <b>${est.srt.toFixed(1)} dB</b> ± ${est.sd.toFixed(1)}. Stops once the uncertainty is ±1.25 dB or less.</p>
        ${trials.length ? trackChart(trials) : ""}
      </aside>
    </div>
  </section>`;
  main.querySelectorAll<HTMLButtonElement>(".key[data-d]").forEach(
    (k) => {
      (k.onclick = () => {
        if (k.dataset.d === "back") typed = typed.slice(0, -1);
        else if (typed.length < 3) typed += k.dataset.d;
        render();
      }),
    );
  document.getElementById("replay").onclick = () => play();
  document.getElementById("submit").onclick = () => submit();
};

const play = async () => {
  await engine().load();
  await engine().playTriplet(target, snr);
};

const submit = () => {
  const { correct } = scoreTriplet(target, typed);
  const trial = { snr, correct };
  trials.push(trial);
  proc.update(trial);
  typed = "";
  if (proc.done()) return finish(st, proc.estimate(), trials, verified || st.virtualSrt !== null);
  target = makeTriplet(rng);
  snr = proc.nextSnr();
  render();
  void next();
};

const onKey = (e: KeyboardEvent) => {
  if (!document.querySelector(".keypad")) return window.removeEventListener("keydown", onKey);
  if (/^[0-9]$/.test(e.key) && e.key !== "7" && typed.length < 3) typed += e.key;
  else if (e.key === "Backspace") typed = typed.slice(0, -1);
  else if (e.key === "Enter" && typed.length === 3) return submit();
  else return;
  render();
};
window.addEventListener("keydown", onKey);

const next = async () => {
  const played = play().catch(() => undefined);
  if (st.virtualSrt === null) return;
  await Promise.race([played, new Promise((r) => setTimeout(r, 1400))]);
  const ok = vrng() < pCorrect(snr, st.virtualSrt, 0.7, 0.02);
  const answer = ok ? target.join("") : target.map((d, k) => (k === 1 ? (d === 9 ? 8 : 9) : d)).join("");
  for (const ch of answer) {
    await new Promise((r) => setTimeout(r, 160));
    typed += ch;
    render();
  }
  await new Promise((r) => setTimeout(r, 250));
  if (document.getElementById("submit")) submit();
};

render();
void next();
}

```

```

function finish(st: CheckState, est: Estimate, trials: Trial[], verified: boolean): void {
  const members = loadMembers();
  const member: Member = { id: `m-${Date.now()}`, name: st.name, estimate: est, takenAt: new Date().toISOString(), device: st.device + (verified ? "" : " (headphones not verified)") };
  const main = app.querySelector("main");
  const sameDevice = members.filter(m => m.device.replace(" (headphones not verified)", "") === st.device);
  main.innerHTML = `<section class="panel">
    <p class="eyebrow">Done in ${trials.length} triplets</p>
    <h2>${esc(st.name)}, your speech-in-noise threshold is <span class="big">${est.srt.toFixed(1)} dB SNR</span></h2>
    <p class="lead">That's the noise level where you'd catch about half of the digit triplets. <b>Lower (more negative) = better at hearing in noise.</b> 90% credible interval: ${est.lo90.toFixed(1)} to ${est.hi90.toFixed(1)} dB.</p>
    <div class="grid2"><div>${trackChart(trials)}<p class="fine">Green = got all three digits, red = missed. The engine moved the noise to where your answers were most informative.</p></div>
    <div class="card"><h3>What does this mean?</h3><p>On its own, not much yet – and we won't pretend otherwise. Your earbuds and volume shift this number by an unknown amount, so Hearsay has <b>no fake "normal range"</b>.</p><p>What <i>is</i> meaningful: comparing people who used <b>this same device</b> (${sameDevice.length} saved so far), and comparing yourself over time.</p>
    ${verified ? "" : `<p class="notice">Headphones were not verified – treat this result with extra caution.</p>`}</div>
    <div class="row"><button class="btn primary" id="save">Save to family board</button><a class="btn" href="#/check">Next person</a><button class="link" id="discard">Discard</button></div>
    <p class="fine">${DISCLAIMER}</p>
  </section>`;
  document.getElementById("save")!.onclick = () => {
    saveMembers([...members, member]);
    location.hash = "#/family";
  };
  document.getElementById("discard")!.onclick = () => (location.hash = "#/");
}

function familyView(): void {
  const members = loadMembers();
  const devices = [...new Set(members.map((m) => m.device.replace(" (headphones not verified)", "")))]
  const groups = devices.map((d) => ({ device: d, rows: compareGroup(members.filter((m) => m.device.replace(" (headphones not verified)", "") === d) }));
  const verdictText: Record<string, string> = {
    harder: "Found it noticeably harder than others on this device. Worth mentioning to a doctor or getting a professional hearing test.",
    easier: "Did noticeably better than others on this device.",
    similar: "Similar to the others on this device.",
    "too-few": "Add at least one more person on this device to compare.",
  };
  app.innerHTML = shell(
    "family",
    `<section class="panel">
      <p class="eyebrow">Same device, same headphones, same volume</p>
      <h2>Family board</h2>
      <p class="lead">Home hearing checks can't calibrate your earbuds. Hearsay sidesteps that: everyone tested on the same setup is compared with the <b>median of everyone else</b>, so the device offset cancels. A gap is only flagged if it's larger than <b>3 dB</b> and larger than the measurement uncertainty.</p>
      ${
        members.length
          ? groups
            .map(
              (g) => `<div class="group"><h3>${ICONS.headphones} ${esc(g.device)}</h3>${familyChart(g.rows)}
              <ul class="verdicts">${g.rows
                .map(
                  (r) => `<li class="${r.verdict}"><b>${esc(r.member.name)}</b> – ${r.member.estimate.srt.toFixed(1)} dB${Number.isFinite(r.delta) ? ` (${r.delta} >= 0 ? "+" : "" )` : ""} vs. others` : ""}. ${verdictText[r.verdict]}</li>
                )
                .join("")}</ul></div>`,
            )
            .join("")
          : `<div class="empty"><p>No one yet. Take the check, pass the headphones to the next person, repeat.</p></div>`
      }
    </section>`,
  );
  document.getElementById("demo")!.onclick = () => {
    saveMembers([...loadMembers().filter((m) => !m.id.startsWith("demo-")), ...demoFamily()]);
    familyView();
  };
  document.getElementById("clear")?.addEventListener("click", () => {
    saveMembers([]);
    familyView();
  });
  app.querySelectorAll<HTMLButtonElement>(".del").forEach(
    (b) =>
      (b.onclick = () => {
        saveMembers(loadMembers().filter((m) => m.id !== b.dataset.id));
        familyView();
      })
  );
}

```



```

}

function simulatorView(): void {
  let current = PROFILES[1];
  const render = () => {
    app.innerHTML = shell(
      "simulator",
      `<section class="panel">
        <p class="eyebrow">Empathy mode · illustrative only</p>
        <h2>Hear through their ears</h2>
        <p class="lead">Most hearing loss doesn't make the world quieter – it makes it <b>blurrier</b>. High pitches f
ade first, taking consonants with them. Play the same message both ways.</p>
        <div class="profiles">${PROFILES.slice(1)
          .map((p) => `<button class="chip ${p.id === current.id ? "on" : ""}" data-p="${p.id}">${p.label}</button>`)
          .join("")}</div>
        <div class="grid2"><div>${profileChart(current.loss, AUDIOGRAM_FREQS)}<p class="fine">${current.description} T
hese curves are hand-made teaching examples, not anyone's real audiogram. Filtering is an approximation: real hearing
loss also affects loudness growth and timing, which this does not model.</p></div>
        <div class="card"><h3>"Hi Grandpa! I'm picking you up at 4:15 on Thursday..."</h3>
        <div class="row"><button class="btn" id="orig">> Original</button><button class="btn primary" id="sim">> Thr
ough "${current.label}"</button></div>
        <div class="row"><button class="btn" id="trip">> Digits in noise, through "${current.label}"</button></div>
        <p class="fine">Tip for families: face the person, reduce background noise, and slow down. Shouting mostly m
akes vowels louder – the part they already hear.</p></div></div>
      </section>`,
    );
  };
  app.querySelectorAll<HTMLButtonElement>(".chip").forEach(
    (c) => {
      (c.onclick = () => {
        current = PROFILES.find((p) => p.id === c.dataset.p)!;
        render();
      }),
    );
  document.getElementById("orig")!.onclick = async () => {
    await engine().load();
    await engine().playSentence(null);
  };
  document.getElementById("sim")!.onclick = async () => {
    await engine().load();
    await engine().playSentence(current);
  };
  document.getElementById("trip")!.onclick = async () => {
    await engine().load();
    await engine().playTripletThrough(current, makeTriplet(rng), 0);
  };
  render();
}

function labView(): void {
  app.innerHTML = shell(
    "lab",
    `<section class="panel">
      <p class="eyebrow">Measurable, reproducible, in your browser</p>
      <h2>The Lab: does the AI actually help?</h2>
      <p class="lead">We simulate hundreds of virtual listeners with known thresholds, run each test procedure on them
twice, and measure the error. Seeded, so you get the same numbers every time.</p>
      <div class="row"><label class="inline">Virtual listeners <select id="n"><option>100</option><option selected>400
</option><option>1000</option></select></label>
      <label class="inline">True slope (dB) <select id="slope"><option>0.5</option><option selected>0.7</option><opti
on>1.0</option></select></label>
      <button class="btn primary" id="run">Run simulation</button><a class="btn" href="#/check?virtual=-4">Watch a vir
tual listener take the check</a></div>
      <div id="out"><p class="fine">Press run (takes a couple of seconds).</p></div>
    </section>`,
  );
  const run = () => {
    const out = document.getElementById("out")!;
    out.innerHTML = `<p class="fine">Simulating...</p>`;
    setTimeout(() => {
      const listeners = Number((document.getElementById("n") as HTMLSelectElement).value);
      const trueSlope = Number((document.getElementById("slope") as HTMLSelectElement).value);
      const res: MethodStats[] = Object.values(METHODS).map((m) => simulate(m, { ...DEFAULT_SIM, listeners, trueSlope
}));
      const [s24, s12, bayes] = res;
      out.innerHTML = `<table class="results"><thead><tr><th>Procedure</th><th>Avg. triplets</th><th>RMSE (dB)</th><th>
>Bias (dB)</th><th>Test–retest SD (dB)</th><th>90% interval really contains truth</th></tr></thead><tbody>
        ${res.map((r) => `<tr class="${r === bayes ? "hl" : ""}"><td>${r.method}</td><td>${r.meanTrials.toFixed(1)}</td>
<td>${r.rmse.toFixed(2)}</td><td>${r.bias} >= 0 ? "+" : ""</td><td>${r.bias.toFixed(2)}</td><td>${r.retestSd.toFixed(2)}</td>
<td>${(r.coverage90 * 100).toFixed(0)}%</td></tr>`).join("")}
      </tbody></table>
      <div class="grid3 stats"><div class="stat"><b>${((1 - bayes.rmse / s12.rmse) * 100).toFixed(0)}%</b><span>lower
error than a staircase of the same length</span></div>
      <div class="stat"><b>${((1 - bayes.meanTrials / s24.meanTrials) * 100).toFixed(0)}%</b><span>fewer triplets than
the classic 24-trial track</span></div>
      <div class="stat"><b>${(bayes.coverage90 * 100).toFixed(0)}%</b><span>of Hearsay's "90%" intervals contained the
truth (staircase: ${(s24.coverage90 * 100).toFixed(0)}%</span></div></div>
    `;
  };
}

```

```

    <div class="grid2"><div><h3>Error distribution – Hearsay</h3>${histogram(bayes.errors, -5, 5, 20, "bar b")}</div>
    </div><h3>Error distribution – 12-trial staircase</h3>${histogram(sl2.errors, -5, 5, 20, "bar s")}</div></div>
    <p class="fine">Trade-off, stated plainly: the full 24-trial staircase is still slightly more precise (RMSE ${s2
4.rmse.toFixed(2)} vs ${bayes.rmse.toFixed(2)} dB) – it just takes twice as long and its error bars are over-confiden
t. Simulated listeners follow a logistic psychometric function with 2% lapses; real people also get tired, distracted
and learn, which this does not capture.</p>`;
    }, 30);
  });
  document.getElementById("run")!.onclick = run;
}

function aboutView(): void {
  app.innerHTML = shell(
    "about",
    `<section class="panel prose">
      <p class="eyebrow">Honesty page</p>
      <h2>What's real, what's synthetic, what's missing</h2>
      <h3>What Hearsay is</h3><p>A browser-based educational self-check of <b>speech understanding in noise</b>, using
a digits-in-noise design (three spoken digits in speech-shaped noise, antiphase presentation), a Bayesian adaptive pr
cedure, and same-device family comparison.</p>
      <h3>What it is not</h3><ul><li>Not a diagnosis, screening device, or substitute for an audiologist. It has <b>no
t been clinically validated</b>.</li><li>Not calibrated: without a calibrated headphone and level, absolute scores are
not comparable across devices. That's why there's no "normal range" on the results page.</li><li>Not suitable for chil
dren, people with tinnitus flare-ups from sound, or anyone for whom sound exposure is uncomfortable.</li></ul>
      <h3>Data provenance</h3><ul><li><b>Speech:</b> nine digit words and one sentence synthesized with the ElevenLabs
text-to-speech API (premade voice "Alice"). Generated by <code>scripts/make-stimuli.ts</code>.</li><li><b>Noise:</b> s
tationary noise computed locally from the digits' long-term spectrum (random phase, seeded). No third-party audio.</li>
<li><b>Validation data:</b> 100% synthetic – virtual listeners simulated in <code>src/engine/simulate.ts</code>. No h
uman-subject data was collected.</li><li><b>Demo family:</b> invented names and numbers, labelled "(synthetic)".</li><
li><b>Your results:</b> stored only in this browser's localStorage. No analytics, no server.</li></ul>
      <h3>Known limitations</h3><ul><li>Psychometric slope and lapse rate are assumptions; the engine marginalizes ove
r three slopes to be robust, and the Lab shows how accuracy changes when the true slope differs.</li><li>Synthetic spe
ech is not the same as recorded clinical stimuli; digits were RMS-equalized but not individually intelligibility-balan
ced in noise.</li><li>The hear-through simulator is a filter-based teaching approximation.</li><li>Family comparison a
ssumes everyone keeps the same device, headphones and volume.</li></ul>
      <h3>Built with</h3><p>TypeScript, Vite, Web Audio API, Vitest. Solo student project for the UnivaBio 2026 hackat
hon.</p>
    </section>`,
  );
}

window.addEventListener("hashchange", route);
route();

```

## src/storage.ts

```

import type { Member } from "./engine/family";

const KEY = "hearsay.members.v1";

export function loadMembers(): Member[] {
  try {
    const raw = localStorage.getItem(KEY);
    return raw ? (JSON.parse(raw) as Member[]) : [];
  } catch {
    return [];
  }
}

export function saveMembers(members: Member[]): void {
  localStorage.setItem(KEY, JSON.stringify(members));
}

/** Clearly labelled synthetic demo family, generated from the simulator – not real people. */
export function demoFamily(): Member[] {
  const now = new Date().toISOString();
  const mk = (id: string, name: string, srt: number, sd: number, trials: number): Member => ({
    id,
    name,
    estimate: { srt, sd, lo90: srt - 1.645 * sd, hi90: srt + 1.645 * sd, trials },
    takenAt: now,
    device: "Demo laptop + wired earbuds (synthetic)",
  });
  return [
    mk("demo-sharon", "Sharon (synthetic)", -11.2, 1.1, 12),
    mk("demo-mom", "Mom (synthetic)", -10.4, 1.2, 11),
    mk("demo-dad", "Dad (synthetic)", -9.6, 1.2, 12),
    mk("demo-grandpa", "Grandpa (synthetic)", -3.1, 1.3, 13),
  ];
}

```

## src/ui/charts.ts

```
import type { Comparison } from "../engine/family";
import type { Trial } from "../engine/types";

const W = 560;

function esc(s: string): string {
  return s.replace(/[\&<>"]/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" })[c!]);
}

export function posteriorChart(marginal: { srt: number; p: number }[], est?: { lo90: number; hi90: number; srt: number }): string {
  const H = 150;
  const xMin = marginal[0].srt;
  const xMax = marginal[marginal.length - 1].srt;
  const pMax = Math.max(...marginal.map((m) => m.p), 1e-9);
  const x = (v: number) => 30 + ((v - xMin) / (xMax - xMin)) * (W - 50);
  const y = (p: number) => H - 25 - (p / pMax) * (H - 45);
  const path = marginal.map((m, i) => `${i ? "L" : "M"}${x(m.srt).toFixed(1)},${y(m.p).toFixed(1)}`).join("");
  const area = `${path}L${x(xMax)},${H - 25}L${x(xMin)},${H - 25}Z`;
  const ticks = [];
  for (let t = Math.ceil(xMin / 5) * 5; t <= xMax; t += 5)
    ticks.push(`<line x1="${x(t)}" x2="${x(t)}" y1="${H - 25}" y2="${H - 20}" class="axis"/><text x="${x(t)}" y="${H - 6}" class="tick">${t}</text>`);
  const band = est
    ? `<rect x="${x(est.lo90)}" y="10" width="${Math.max(2, x(est.hi90) - x(est.lo90))}" height="${H - 35}" class="band"/><line x1="${x(est.srt)}" x2="${x(est.srt)}" y1="10" y2="${H - 25}" class="mean"/>`
    : "";
  return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Belief about your speech reception threshold">
    ${band}<path d="${area}" class="area"/><path d="${path}" class="line"/>
    <line x1="30" x2="${W - 20}" y1="${H - 25}" y2="${H - 25}" class="axis"/>${ticks.join("")}
    <text x="${W - 20}" y="${H - 30}" class="tick" text-anchor="end">SNR (dB) </text>
  </svg>`;
}

export function trackChart(trials: Trial[]): string {
  const H = 130;
  const n = Math.max(12, trials.length);
  const yMin = -22;
  const yMax = 10;
  const x = (i: number) => 44 + (i / (n - 1 || 1)) * (W - 64);
  const y = (v: number) => 10 + ((yMax - v) / (yMax - yMin)) * (H - 35);
  const pts = trials.map((t, i) => `<circle cx="${x(i)}" cy="${y(t.snr)}" r="5" class="${t.correct ? "ok" : "miss"}"><title>Trial ${i + 1}: ${t.snr} dB, ${t.correct ? "correct" : "missed"}</title></circle>`);
  const line = trials.map((t, i) => `${i ? "L" : "M"}${x(i)},${y(t.snr)}).join("");
  const grid = [-20, -10, 0, 10].map((v) => `<line x1="36" x2="${W - 20}" y1="${y(v)}" y2="${y(v)}" class="grid"/><text x="30" y="${y(v) + 4}" class="tick" text-anchor="end">${v}</text>`);
  return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Trial-by-trial SNR track">${grid.join("")}
  <path d="${line}" class="track"/>${pts.join("")}
  <text x="${W - 20}" y="${H - 6}" class="tick" text-anchor="end">trial </text></svg>`;
}

export function familyChart(rows: Comparison[]): string {
  if (!rows.length) return "";
  const rowH = 40;
  const H = rows.length * rowH + 62;
  const all = rows.flatMap((r) => [r.member.estimate.lo90, r.member.estimate.hi90]);
  const xMin = Math.floor(Math.min(...all, -14) / 2) * 2;
  const xMax = Math.ceil(Math.max(...all, 0) / 2) * 2;
  const L = 190;
  const x = (v: number) => L + ((v - xMin) / (xMax - xMin)) * (W - L - 20);
  const ticks = [];
  for (let t = xMin; t <= xMax; t += 2) ticks.push(`<line x1="${x(t)}" x2="${x(t)}" y1="10" y2="${H - 48}" class="grid"/><text x="${x(t)}" y="${H - 32}" class="tick">${t}</text>`);
  const items = rows.map((r, i) => {
    const cy = 14 + i * rowH + rowH / 2;
    const e = r.member.estimate;
    return `<text x="${L - 12}" y="${cy + 4}" class="label" text-anchor="end">${esc(r.member.name)}</text>
    <line x1="${x(e.lo90)}" x2="${x(e.hi90)}" y1="${cy}" y2="${cy}" class="ci ${r.verdict}"/>
    <circle cx="${x(e.srt)}" cy="${cy}" r="7" class="dot ${r.verdict}"><title>${e.srt.toFixed(1)} dB (90% interval ${e.lo90.toFixed(1)} to ${e.hi90.toFixed(1)})</title></circle>`;
  });
  return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Family comparison">${ticks.join("")}${items.join("")}
  <text x="${(L + W - 20) / 2}" y="${H - 10}" class="tick">← copes better in noise · threshold (dB SNR) · needs clearer speech </text></svg>`;
}

export function histogram(values: number[], lo: number, hi: number, bins: number, cls: string): string {
  const H = 110;
  const counts = new Array(bins).fill(0);
  for (const v of values) counts[Math.max(0, Math.min(bins - 1, Math.floor(((v - lo) / (hi - lo)) * bins)))]++;
  const max = Math.max(...counts, 1);
}
```

```

    const bw = (W - 50) / bins;
    const bars = counts.map((c, i) => `<rect x="${30 + i * bw + 1}" y="${H - 25 - (c / max) * (H - 35)}" width="${bw - 2}" height="${(c / max) * (H - 35)}" class="${cls}" />`);
    const ticks = [lo, (lo + hi) / 2, hi].map((t) => `<text x="${30 + ((t - lo) / (hi - lo)) * (W - 50)}" y="${H - 8}" class="tick">${t}</text>`);
    return `<svg viewBox="0 0 ${W} ${H}" class="chart small">${bars.join("")}<line x1="30" x2="${W - 20}" y1="${H - 25}" y2="${H - 25}" class="axis">${ticks.join("")}</svg>`;
}

export function profileChart(loss: number[], freqs: readonly number[]): string {
    const H = 150;
    const x = (i: number) => 50 + (i / (freqs.length - 1)) * (W - 80);
    const y = (db: number) => 15 + (db / 80) * (H - 45);
    const path = loss.map((l, i) => `${i} ? "L" : "M"${x(i)}, ${y(l)}'`).join("");
    const grid = [0, 20, 40, 60, 80].map((d) => `<line x1="50" x2="${W - 30}" y1="${y(d)}" y2="${y(d)}" class="grid"/>`);
    const fl = freqs.map((f, i) => `<text x="${x(i)}" y="${H - 8}" class="tick">${f} >= 1000 ? f / 1000 + "k" : f</text>`);
    const pts = loss.map((l, i) => `<circle cx="${x(i)}" cy="${y(l)}" r="5" class="dot harder"/>`);
    return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Illustrative attenuation by frequency">${grid.join("")}${fl.join("")}<path d="${path}" class="line"/>${pts.join("")}
    <text x="${W - 30}" y="${H - 22}" class="tick" text-anchor="end">${d} ? -d : 0</text>`;
    const t = `<text x="4" y="${H - 22}" class="tick" text-anchor="start">dB</text>`;
    return `<svg>`;
}

export { esc };

```

## src/ui/icons.ts

```

const svg = (body: string) =>
    `<svg class="icon" viewBox="0 0 24 24" width="22" height="22" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" aria-hidden="true">${body}</svg>`;

export const ICONS = {
    ear: svg(`<path d="M6 8.5a6 6 0 1 1 12 0c0 3.5-3 4.5-3 7.5a3 3 0 0 1-6 0" />`),
    target: svg(`<circle cx="12" cy="12" r="9" />`),
    headphones: svg(`<path d="M4 15v-3a8 8 0 0 1 16 0v3" />`),
    rect: svg(`<rect x="3" y="14" width="4" height="7" rx="1.5" />`),
    rect2: svg(`<rect x="17" y="14" width="4" height="7" rx="1.5" />`),
};

```

## src/style.css

```

:root {
    --bg: #fbf7f1;
    --ink: #1f2430;
    --muted: #5f6675;
    --card: #ffffff;
    --line: #e7e0d5;
    --accent: #e0573b;
    --accent2: #2f6f73;
    --ok: #2f9e6d;
    --miss: #d8443a;
    --soft: #fdeee8;
    font-family: "Inter", ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
    color: var(--ink);
    background: var(--bg);
}

* { box-sizing: border-box; }
body { margin: 0; }
a { color: var(--accent2); }
.top { display: flex; align-items: center; justify-content: space-between; padding: 14px 32px; border-bottom: 1px solid var(--line); background: rgba(251, 247, 241, 0.9); position: sticky; top: 0; backdrop-filter: blur(6px); z-index: 5; flex-wrap: wrap; gap: 8px; }
.brand { font-weight: 800; font-size: 1.25rem; text-decoration: none; color: var(--ink); }
nav { display: flex; gap: 4px; flex-wrap: wrap; }
nav a { text-decoration: none; color: var(--muted); padding: 8px 12px; border-radius: 999px; font-weight: 600; font-size: 0.95rem; }
nav a.active, nav a:hover { background: var(--soft); color: var(--accent); }
main { max-width: 1080px; margin: 0 auto; padding: 32px 24px 16px; }
footer { max-width: 1080px; margin: 0 auto; padding: 16px 24px 40px; color: var(--muted); font-size: 0.85rem; border-top: 1px solid var(--line); }
h1 { font-size: clamp(2rem, 4.5vw, 3.4rem); line-height: 1.05; margin: 8px 0 16px; letter-spacing: -0.02em; }
h2 { font-size: 1.8rem; margin: 4px 0 12px; letter-spacing: -0.01em; }
h3 { margin: 0 0 8px; }
.eyebrow { text-transform: uppercase; letter-spacing: 0.12em; font-size: 0.75rem; font-weight: 700; color: var(--accent); margin: 0; }
.lead { font-size: 1.12rem; line-height: 1.6; color: #333a48; max-width: 780px; }
.fine { font-size: 0.85rem; color: var(--muted); line-height: 1.5; }
.hero { padding: 24px 0 8px; }
.cta, .row { display: flex; gap: 12px; flex-wrap: wrap; align-items: center; margin: 16px 0; }
.btn, .key, .chip, .link { font-family: inherit; }
.btn { display: inline-flex; align-items: center; gap: 6px; padding: 12px 20px; border-radius: 12px; border: 1.5px solid var(--line); background: var(--card); color: var(--ink); font-weight: 700; font-size: 1rem; cursor: pointer; text-d

```

```

ecoration: none; transition: transform 0.08s, box-shadow 0.15s; }
.btn:hover { transform: translateY(-1px); box-shadow: 0 4px 14px rgba(0, 0, 0, 0.07); }
.btn.primary { background: var(--accent); border-color: var(--accent); color: white; }
.btn.disabled { opacity: 0.4; cursor: default; transform: none; }
.link { background: none; border: 0; color: var(--muted); text-decoration: underline; cursor: pointer; font-size: 0.9rem; }
.grid3 { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 16px; margin: 24px 0; }
.grid2 { display: grid; grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)); gap: 20px; margin: 16px 0; align-items: start; }
.card, .panel { background: var(--card); border: 1px solid var(--line); border-radius: 18px; padding: 22px; }
.panel { padding: 32px; box-shadow: 0 10px 40px rgba(60, 40, 20, 0.05); }
.panel.narrow { max-width: 620px; margin: 0 auto; }
.card p { line-height: 1.55; color: #3a4150; }
.how { margin: 32px 0; }
.steps { list-style: none; padding: 0; display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 12px; counter-reset: s; }
.steps li { counter-increment: s; background: var(--card); border: 1px solid var(--line); border-radius: 14px; padding: 16px 16px 16px 56px; position: relative; display: flex; flex-direction: column; gap: 4px; }
.steps li::before { content: counter(s); position: absolute; left: 16px; top: 16px; width: 28px; height: 28px; border-radius: 50%; background: var(--accent2); color: white; display: grid; place-items: center; font-weight: 800; }
.steps span { color: var(--muted); font-size: 0.92rem; }
label { display: flex; flex-direction: column; gap: 6px; font-weight: 600; margin: 14px 0; }
label.inline { flex-direction: row; align-items: center; margin: 0; }
input, select { font: inherit; padding: 12px 14px; border-radius: 10px; border: 1.5px solid var(--line); background: #fffdfa; }
input[type="range"] { flex: 1; min-width: 200px; accent-color: var(--accent); padding: 0; }
.notice { background: #fff6d8; border: 1px solid #f0dc97; padding: 10px 14px; border-radius: 10px; font-size: 0.92rem; }
.choices { display: flex; gap: 12px; margin: 20px 0; }
.choice { width: 72px; height: 72px; justify-content: center; font-size: 1.6rem; }
.testhead { display: flex; justify-content: space-between; align-items: center; gap: 16px; }
.meter { flex: 0 0 200px; height: 8px; background: var(--line); border-radius: 99px; overflow: hidden; }
.meter span { display: block; height: 100%; background: var(--accent2); transition: width 0.3s; }
.testgrid { display: grid; grid-template-columns: minmax(280px, 380px) 1fr; gap: 32px; margin-top: 16px; }
@media (max-width: 800px) { .testgrid { grid-template-columns: 1fr; } }
.display { display: flex; gap: 10px; margin: 16px 0; }
.display span { width: 72px; height: 84px; border-radius: 14px; border: 2px dashed var(--line); display: grid; place-items: center; font-size: 2.4rem; font-weight: 800; color: #c8bfb2; }
.display span.filled { border-style: solid; border-color: var(--accent2); color: var(--ink); background: #f1f8f7; }
.keypad { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; max-width: 300px; }
.key { height: 60px; border-radius: 14px; border: 1.5px solid var(--line); background: #fff; font-size: 1.4rem; font-weight: 700; cursor: pointer; }
.key:hover { background: var(--soft); }
.key.alt { color: var(--muted); }
.key.submit { background: var(--accent); color: #fff; border-color: var(--accent); }
.key.submit.disabled { opacity: 0.35; }
.belief { background: #f7faf9; border: 1px solid #dbe9e7; border-radius: 16px; padding: 18px; }
.chart { width: 100%; height: auto; display: block; margin: 8px 0; }
.chart .area { fill: rgba(47, 111, 115, 0.18); }
.chart .line { fill: none; stroke: var(--accent2); stroke-width: 2.5; }
.chart .band { fill: rgba(224, 87, 59, 0.12); }
.chart .mean { stroke: var(--accent); stroke-width: 2; stroke-dasharray: 4 3; }
.chart .axis { stroke: #b8b0a3; }
.chart .grid { stroke: #eee8de; }
.chart .tick { font-size: 11px; fill: var(--muted); text-anchor: middle; }
.chart .label { font-size: 13px; font-weight: 700; fill: var(--ink); }
.chart .track { fill: none; stroke: #c9c1b4; stroke-width: 1.5; }
.chart circle.ok { fill: var(--ok); }
.chart circle.miss { fill: var(--miss); }
.chart .ci { stroke-width: 6; stroke-linecap: round; stroke: #cfd8d7; }
.chart .ci.harder { stroke: #f4c2b6; }
.chart .dot { fill: var(--accent2); stroke: white; stroke-width: 2; }
.chart .dot.harder { fill: var(--accent); }
.chart .dot.easier { fill: var(--ok); }
.chart .bar.b { fill: var(--accent2); }
.chart .bar.s { fill: #b9ada0; }
.big { color: var(--accent); white-space: nowrap; }
.group { margin: 20px 0; padding: 18px; border: 1px solid var(--line); border-radius: 14px; background: #fffdfa; }
.verdicts { list-style: none; padding: 0; margin: 8px 0 0; display: grid; gap: 8px; }
.verdicts li { padding: 10px 14px; border-radius: 10px; background: #f4f2ee; line-height: 1.45; }
.verdicts li.harder { background: var(--soft); border-left: 4px solid var(--accent); }
.verdicts li.easier { background: #eaf6f0; }
.verdicts .del { float: right; }
.empty { padding: 30px; text-align: center; border: 2px dashed var(--line); border-radius: 14px; color: var(--muted); }
.profiles { display: flex; gap: 8px; flex-wrap: wrap; margin: 12px 0; }
.chip { padding: 8px 16px; border-radius: 99px; border: 1.5px solid var(--line); background: #fff; cursor: pointer; font-weight: 700; }
.chip.on { background: var(--accent2); color: #fff; border-color: var(--accent2); }
table.results { width: 100%; border-collapse: collapse; margin: 16px 0; font-size: 0.95rem; }
.results th, .results td { padding: 10px 12px; border-bottom: 1px solid var(--line); text-align: right; }
.results th:first-child, .results td:first-child { text-align: left; }
.results tr.hl { background: #f1f8f7; font-weight: 700; }
.stat { background: var(--soft); border-radius: 14px; padding: 18px; display: flex; flex-direction: column; gap: 4px; }
.stat b { font-size: 2.2rem; color: var(--accent); }
.stat span { color: #4a4f5c; }

```

```
.prose { max-width: 820px; margin: 0 auto; line-height: 1.6; }
.prose li { margin: 4px 0; }
code { background: #f1f3f4; padding: 1px 6px; border-radius: 6px; font-size: 0.9em; }
.icon { vertical-align: -4px; color: var(--accent); margin-right: 4px; }
.brand .icon { width: 26px; height: 26px; vertical-align: -6px; }
```

## scripts/make-stimuli.ts

```
/**
 * Generates the digit tokens, demo sentence and speech-shaped noise in public/stimuli.
 * Speech is synthesized with the ElevenLabs text-to-speech API (requires ELEVENLABS_API_KEY);
 * the noise is derived locally from the digit tokens' long-term spectrum.
 */
import { mkdirSync, writeFileSync } from "node:fs";
import { join } from "node:path";
import { normalizeRms, speechShapedNoise, trimSilence } from "../src/dsp/stimuli";
import { encodeWav } from "../wav";

const SR = 24000;
const VOICE_ID = process.env.HEARSAY_VOICE_ID ?? "21m00Tcm4TlvDq8ikWAM";
const MODEL = "eleven_multilingual_v2";
const OUT = join(process.cwd(), "public", "stimuli");
export const DIGIT_WORDS: Record<number, string> = { 0: "zero", 1: "one", 2: "two", 3: "three", 4: "four", 5: "five", 6: "six", 8: "eight", 9: "nine" };
const SENTENCE = "Hi Grandpa! I'm picking you up at four fifteen on Thursday. Bring your blue jacket, and don't forget the pharmacy slip for Doctor Patel.";

async function tts(text: string): Promise<Float32Array> {
  const key = process.env.ELEVENLABS_API_KEY;
  if (!key) throw new Error("ELEVENLABS_API_KEY is not set");
  const res = await fetch(`https://api.elevenlabs.io/v1/text-to-speech/${VOICE_ID}?output_format=pcm_24000`, {
    method: "POST",
    headers: { "xi-api-key": key, "content-type": "application/json" },
    body: JSON.stringify({ text, model_id: MODEL, voice_settings: { stability: 0.8, similarity_boost: 0.7 } }),
  });
  if (!res.ok) throw new Error(`TTS failed for "${text}": ${res.status} ${await res.text()}`);
  const buf = Buffer.from(await res.arrayBuffer());
  const out = new Float32Array(buf.length / 2);
  for (let i = 0; i < out.length; i++) out[i] = buf.readInt16LE(i * 2) / 32768;
  return out;
}

async function main(): Promise<void> {
  mkdirSync(OUT, { recursive: true });
  const tokens: Float32Array[] = [];
  const manifest: Record<string, number> = {};
  for (const [digit, word] of Object.entries(DIGIT_WORDS)) {
    const raw = await tts(`${word}.`);
    const tok = normalizeRms(trimSilence(raw, SR));
    tokens.push(tok);
    manifest[digit] = tok.length / SR;
    writeFileSync(join(OUT, `digit-${digit}.wav`), encodeWav(tok, SR));
    console.log(`digit ${digit}: ${tok.length / SR}.toFixed(3)} s`);
  }
  const noise = speechShapedNoise(tokens, 1 << 17, 20261006);
  writeFileSync(join(OUT, "noise.wav"), encodeWav(noise, SR));
  const sentence = normalizeRms(trimSilence(await tts(SENTENCE), SR, -45, 60));
  writeFileSync(join(OUT, "sentence.wav"), encodeWav(sentence, SR));
  writeFileSync(
    join(OUT, "manifest.json"),
    JSON.stringify({ sampleRate: SR, voiceId: VOICE_ID, model: MODEL, generatedAt: new Date().toISOString(), digitDurations: manifest, sentence: SENTENCE }, null, 2) + "\n",
  );
}

main().catch((e) => {
  console.error(e);
  process.exit(1);
});
```

## scripts/simulate.ts

```
import { writeFileSync } from "node:fs";
import { DEFAULT_SIM, METHODS, simulate } from "../src/engine/simulate";

const slopes = [0.5, 0.7, 1.0];
const rows: Record<string, unknown>[] = [];
for (const trueSlope of slopes) {
  for (const [key, make] of Object.entries(METHODS)) {
    const s = simulate(make, { ...DEFAULT_SIM, trueSlope });
    rows.push({ key, method: s.method, trueSlope, rmse: s.rmse, bias: s.bias, meanTrials: s.meanTrials, coverage90: s.coverage90, retestSd: s.retestSd });
    console.log(
```

```
`${s.method.padEnd(22)} slope=${trueSlope} RMSE=${s.rmse.toFixed(2)} dB bias=${s.bias.toFixed(2)} trials=${s.meanTrials.toFixed(1)} cover90=${(s.coverage90 * 100).toFixed(0)}% retestSD=${s.retestSd.toFixed(2)}`,
);
}
}
writeFileSync("docs/validation.json", JSON.stringify({ config: DEFAULT_SIM, slopes, rows }, null, 2) + "\n");
```

## scripts/wav.ts

```
export function encodeWav(x: Float32Array, sampleRate: number): Buffer {
  const buf = Buffer.alloc(44 + x.length * 2);
  buf.write("RIFF", 0);
  buf.writeUInt32LE(36 + x.length * 2, 4);
  buf.write("WAVE", 8);
  buf.write("fmt ", 12);
  buf.writeUInt32LE(16, 16);
  buf.writeUInt16LE(1, 20);
  buf.writeUInt16LE(1, 22);
  buf.writeUInt32LE(sampleRate, 24);
  buf.writeUInt32LE(sampleRate * 2, 28);
  buf.writeUInt16LE(2, 32);
  buf.writeUInt16LE(16, 34);
  buf.write("data", 36);
  buf.writeUInt32LE(x.length * 2, 40);
  for (let i = 0; i < x.length; i++) {
    const v = Math.max(-1, Math.min(1, x[i]));
    buf.writeInt16LE(Math.round(v * 32767), 44 + i * 2);
  }
  return buf;
}
```

## tests/dsp.test.ts

```
import { readFileSync } from "node:fs";
import { describe, expect, it } from "vitest";
import { dbToGain, fft, rms } from "../src/dsp/fft";
import { normalizeRms, speechShapedNoise, trimSilence } from "../src/dsp/stimuli";
import { mixTriplet } from "../src/audio/mix";
import { lossAt, PROFILES } from "../src/audio/hearloss";

function readWav(path: string): Float32Array {
  const b = readFileSync(path);
  const n = b.readUInt32LE(40) / 2;
  const out = new Float32Array(n);
  for (let i = 0; i < n; i++) out[i] = b.readInt16LE(44 + i * 2) / 32768;
  return out;
}

describe("FFT", () => {
  it("round-trips", () => {
    const re = Float64Array.from({ length: 64 }, (_, i) => Math.sin(i) + 0.3 * i);
    const orig = re.slice();
    const im = new Float64Array(64);
    fft(re, im);
    fft(re, im, true);
    re.forEach((v, i) => expect(v).toBeCloseTo(orig[i], 9));
  });
  it("puts a pure tone in the right bin", () => {
    const n = 256;
    const re = Float64Array.from({ length: n }, (_, i) => Math.cos((2 * Math.PI * 10 * i) / n));
    const im = new Float64Array(n);
    fft(re, im);
    const mags = [...re].map((r, i) => Math.hypot(r, im[i]));
    expect(mags.indexOf(Math.max(...mags.slice(0, n / 2))))>= 10;
  });
  it("rejects non power-of-two lengths", () => {
    expect(() => fft(new Float64Array(3), new Float64Array(3))).toThrow();
  });
});

describe("stimulus processing", () => {
  it("trims silence and normalizes RMS", () => {
    const x = new Float32Array(1000);
    for (let i = 400; i < 600; i++) x[i] = Math.sin(i);
    const t = trimSilence(x, 1000, -40, 0);
    expect(t.length).toBeLessThan(220);
    expect(rms(normalizeRms(t, 0.1))).toBeCloseTo(0.1, 5);
  });
  it("makes noise with the speech's spectral tilt", () => {
    const low = Float32Array.from({ length: 1024 }, (_, i) => Math.sin((2 * Math.PI * 20 * i) / 1024));
    const noise = speechShapedNoise([low], 4096, 1);
    expect(noise.length).toBe(4096);
    expect(rms(noise)).toBeCloseTo(0.1, 4);
    const re = Float64Array.from(noise);
```



```

    const im = new Float64Array(4096);
    fft(re, im);
    const band = (a: number, b: number) => {
      let s = 0;
      for (let k = a; k < b; k++) s += re[k] ** 2 + im[k] ** 2;
      return s;
    };
    expect(band(60, 100)).toBeGreaterThan(band(1000, 1040) * 100);
  });
});

describe("shipped stimuli", () => {
  it("are RMS-equalized digits and matching noise", () => {
    for (const d of [0, 1, 2, 3, 4, 5, 6, 8, 9]) {
      const x = readWav(`public/stimuli/digit-${d}.wav`);
      expect(x.length / 24000).toBeGreaterThan(0.2);
      expect(x.length / 24000).toBeLessThan(1);
      expect(rms(x)).toBeCloseTo(0.1, 2);
    }
    expect(rms(readWav("public/stimuli/noise.wav"))).toBeCloseTo(0.1, 2);
  });
});

describe("triplet mixing", () => {
  const tok = Float32Array.from({ length: 200 }, (_, i) => 0.1 * Math.sin(i / 3) * Math.SQRT2);
  const noise = Float32Array.from({ length: 997 }, (_, i) => 0.1 * Math.sin(i * 1.7) * Math.SQRT2);
  it("sets the requested SNR", () => {
    for (const snr of [-10, 0, 6]) {
      const m = mixTriplet([tok, tok, tok], new Float32Array(997), { sampleRate: 1000, snr, antiphasic: false, noiseGain: 0.1 });
      const seg = m.left.slice(Math.round(m.onsets[0] * 1000), Math.round(m.onsets[0] * 1000) + 200);
      expect(20 * Math.log10(rms(seg) / (0.1 * 0.1))).toBeCloseTo(snr, 1);
    }
  });
  it("inverts speech but not noise in the right ear when antiphasic", () => {
    const m = mixTriplet([tok, tok, tok], noise, { sampleRate: 1000, snr: 0, antiphasic: true, noiseGain: 0.1 });
    for (let i = 0; i < 400; i++) expect(m.left[i]).toBeCloseTo(m.right[i], 6);
    const on = Math.round(m.onsets[1] * 1000) + 50;
    const sum = m.left[on] + m.right[on];
    const nz = noise[on % noise.length] * 0.1;
    expect(sum).toBeCloseTo(2 * nz, 5);
  });
  it("never clips", () => {
    const loud = tok.map((v) => v * 30);
    const m = mixTriplet([loud, loud, loud], noise, { sampleRate: 1000, snr: 10, antiphasic: true });
    expect(Math.max(...m.left.map(Math.abs))).toBeLessThanOrEqual(0.95 + 1e-6);
  });
});

describe("hear-through profiles", () => {
  it("interpolate in log-frequency and slope downward", () => {
    const p = PROFILES.find((x) => x.id === "mild-hf");
    expect(lossAt(p, 250)).toBe(0);
    expect(lossAt(p, 8000)).toBe(45);
    expect(lossAt(p, Math.sqrt(2000 * 4000))).toBeCloseTo(27.5);
    expect(dbToGain(-20)).toBeCloseTo(0.1);
  });
});

```

## tests/engine.test.ts

```

import { describe, expect, it } from "vitest";
import { BayesianDin } from "../src/engine/bayes";
import { Staircase } from "../src/engine/staircase";
import { pCorrect } from "../src/engine/psychometric";
import { mulberry32 } from "../src/engine/rng";
import { DEFAULT_SIM, METHODS, runOnce, simulate } from "../src/engine/simulate";
import { DIGITS, makeTriplet, parseResponse, scoreTriplet } from "../src/engine/triplets";
import { headphonePassed, makeHeadphoneTrial } from "../src/engine/headphones";

describe("psychometric function", () => {
  it("is 50% of (1 - lapse) at the SRT and monotonic", () => {
    expect(pCorrect(-6, -6, 0.7, 0)).toBeCloseTo(0.5);
    expect(pCorrect(-10, -6)).toBeLessThan(pCorrect(-6, -6));
    expect(pCorrect(10, -6, 0.7, 0.03)).toBeLessThanOrEqual(0.97);
  });
});

describe("triplets", () => {
  it("uses three distinct digits from the set and never 7", () => {
    const rng = mulberry32(1);
    for (let i = 0; i < 500; i++) {
      const t = makeTriplet(rng);
      expect(new Set(t).size).toBe(3);
      for (const d of t) expect(DIGITS).toContain(d);
    }
  });
});

```



```

    expect(t).not.toContain(7);
  }
});
it("scores whole-triplet correctness in order", () => {
  expect(scoreTriplet([1, 2, 3], "123")).toEqual({ correct: true, digitsCorrect: 3 });
  expect(scoreTriplet([1, 2, 3], "1 2 4")).toEqual({ correct: false, digitsCorrect: 2 });
  expect(scoreTriplet([1, 2, 3], "321").correct).toBe(false);
  expect(scoreTriplet([1, 2, 3], "1234").correct).toBe(false);
  expect(parseResponse("a1-2 3")).toEqual([1, 2, 3]);
});
});

describe("staircase", () => {
  it("moves down 2 dB after a correct triplet and up after a miss", () => {
    const s = new Staircase();
    expect(s.nextSnr()).toBe(0);
    s.update({ snr: 0, correct: true });
    expect(s.nextSnr()).toBe(-2);
    s.update({ snr: -2, correct: false });
    expect(s.nextSnr()).toBe(0);
  });
  it("runs exactly 24 trials", () => {
    const e = runOnce(METHODS.staircase, -8, 0.7, 0.02, mulberry32(3));
    expect(e.trials).toBe(24);
  });
});

describe("Bayesian procedure", () => {
  it("starts with a broad belief and narrows after informative answers", () => {
    const b = new BayesianDin();
    const sd0 = b.estimate().sd;
    const rng = mulberry32(5);
    for (let i = 0; i < 10; i++) {
      const snr = b.nextSnr();
      b.update({ snr, correct: rng() < pCorrect(snr, -9) });
    }
    expect(b.estimate().sd).toBeLessThan(sd0 / 2);
    const total = b.marginal().reduce((a, m) => a + m.p, 0);
    expect(total).toBeCloseTo(1, 6);
  });
  it("places trials near the current belief, not at the extremes", () => {
    const b = new BayesianDin();
    const first = b.nextSnr();
    expect(first).toBeGreaterThan(-16);
    expect(first).toBeLessThan(0);
  });
  it("moves its estimate toward the listener", () => {
    const good = runOnce(METHODS.bayesian, -12, 0.7, 0.02, mulberry32(11));
    const poor = runOnce(METHODS.bayesian, 0, 0.7, 0.02, mulberry32(11));
    expect(good.srt).toBeLessThan(-9);
    expect(poor.srt).toBeGreaterThan(-3);
  });
  it("respects min/max trial limits", () => {
    const e = runOnce(METHODS.bayesian, -6, 0.7, 0.02, mulberry32(2));
    expect(e.trials).toBeGreaterThanOrEqual(10);
    expect(e.trials).toBeLessThanOrEqual(24);
  });
});

describe("simulation study (the headline claims)", () => {
  const cfg = { ...DEFAULT_SIM, listeners: 200 };
  const s24 = simulate(METHODS.staircase, cfg);
  const s12 = simulate(METHODS.staircase12, cfg);
  const b = simulate(METHODS.bayesian, cfg);

  it("is deterministic for a given seed", () => {
    expect(simulate(METHODS.bayesian, { ...cfg, listeners: 30 }).rmse).toBe(simulate(METHODS.bayesian, { ...cfg, listeners: 30 }).rmse);
  });
  it("beats a staircase of the same length on error", () => {
    expect(b.meanTrials).toBeLessThan(14);
    expect(b.rmse).toBeLessThan(s12.rmse * 0.8);
  });
  it("uses roughly half the trials of the classic 24-trial track", () => {
    expect(b.meanTrials).toBeLessThan(s24.meanTrials * 0.6);
  });
  it("reports 90% intervals that are approximately calibrated", () => {
    expect(b.coverage90).toBeGreaterThan(0.85);
    expect(b.coverage90).toBeGreaterThan(s24.coverage90);
  });
});

describe("headphone check", () => {
  it("never makes the quiet tone the antiphase tone", () => {
    const rng = mulberry32(9);
    for (let i = 0; i < 200; i++) {
      const t = makeHeadphoneTrial(rng);

```

```

    expect(t.quiet).not.toBe(t.antiphase);
    expect([0, 1, 2]).toContain(t.quiet);
    expect([0, 1, 2]).toContain(t.antiphase);
  }
});
it("passes at 5 of 6", () => {
  expect(headphonePassed(5)).toBe(true);
  expect(headphonePassed(4)).toBe(false);
});
});
});

```

## tests/family.test.ts

```

import { describe, expect, it } from "vitest";
import { compareGroup, median, type Member } from "../src/engine/family";
import { demoFamily } from "../src/storage";

const mk = (id: string, srt: number, sd = 1): Member => ({
  id,
  name: id,
  estimate: { srt, sd, lo90: srt - 1.645 * sd, hi90: srt + 1.645 * sd, trials: 12 },
  takenAt: "2026-01-01T00:00:00Z",
  device: "d",
});

describe("family comparison", () => {
  it("computes medians", () => {
    expect(median([3, 1, 2])).toBe(2);
    expect(median([4, 1, 2, 3])).toBe(2.5);
  });
  it("needs at least two people", () => {
    expect(compareGroup([mk("a", -10)])[0].verdict).toBe("too-few");
  });
  it("flags only the clear outlier", () => {
    const rows = compareGroup([mk("a", -11), mk("b", -10), mk("c", -9.5), mk("grandpa", -3)]);
    expect(rows.map((r) => r.verdict)).toEqual(["similar", "similar", "similar", "harder"]);
    expect(rows[3].delta).toBeCloseTo(7);
  });
  it("does not flag small gaps even with tight intervals", () => {
    const rows = compareGroup([mk("a", -10, 0.2), mk("b", -8, 0.2)]);
    expect(rows.every((r) => r.verdict === "similar")).toBe(true);
  });
  it("does not flag larger gaps when both measurements are too uncertain", () => {
    const rows = compareGroup([mk("a", -10, 3), mk("b", -5, 3)]);
    expect(rows.every((r) => r.verdict === "similar")).toBe(true);
  });
  it("is invariant to a shared device offset", () => {
    const base = [mk("a", -11), mk("b", -10), mk("c", -3)];
    const shifted = base.map((m) => mk(m.id, m.estimate.srt + 6.5));
    expect(compareGroup(shifted).map((r) => r.verdict)).toEqual(compareGroup(base).map((r) => r.verdict));
  });
  it("flags Grandpa in the synthetic demo family", () => {
    const rows = compareGroup(demoFamily());
    expect(rows.filter((r) => r.verdict === "harder").map((r) => r.member.name)).toEqual(["Grandpa (synthetic)"]);
  });
});

```